

1. Árvore B

Características:

- Árvore de busca balanceada usada principalmente em sistemas de banco de dados e arquivos de disco.
- Cada nó pode conter múltiplas chaves e filhos.
- Altura pequena mesmo com muitos dados.
- Todos os dados são armazenados nas folhas.

Exemplo :

Inserindo os elementos: 10, 20, 5, 6, 12, 30

Com ordem 3 (máximo 2 chaves por nó):

<https://www.cs.usfca.edu/~galles/visualization/BTree.html>

Remover 10: Substitui pela menor chave da subárvore à direita ou maior da esquerda → reequilibra se necessário.

1. Em uma árvore B de ordem 4, qual o número máximo de filhos que um nó pode ter?
A) 2
B) 3
C) 4
D) 5
2. Qual característica torna a árvore B ideal para sistemas de banco de dados?
A) Simplicidade
B) Alto custo de inserção
C) Baixa altura e acessos a disco otimizados
D) Nenhuma
3. Em uma árvore B de ordem 3, quantas chaves pode ter um nó interno?
A) 1 a 2
B) 2 a 3
C) 1 a 3
D) 0 a 2

4. O que ocorre quando um nó é inserido em um nó já cheio?
 - A) Ignora
 - B) Transforma em folha
 - C) Divide o nó
 - D) Remove a chave
5. Qual alternativa é verdadeira sobre a remoção em árvores B?
 - A) Remove diretamente
 - B) Não há remoção
 - C) Sempre remove da raiz
 - D) Pode exigir fusão de nós

2. Árvore B+

Características:

- Variante da B onde apenas folhas armazenam os dados.
- Nós internos armazenam apenas chaves de direcionamento.
- As folhas são encadeadas sequencialmente (útil para intervalos).
- Muito usada em bancos de dados.

Exemplo de Inserção:

Inserindo 5, 10, 15, 20, 25:

Remover 10 → remove da folha → atualiza os nós internos se necessário.

<https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>

1. Onde são armazenadas as chaves em uma árvore B+?
 - A) Apenas em nós internos
 - B) Apenas nas folhas
 - C) Em todos os nós
 - D) Somente na raiz
2. Qual vantagem principal da B+ sobre a B?
 - A) Menos espaço
 - B) Inserção mais rápida
 - C) Busca eficiente por intervalo
 - D) Nós com mais filhos

3. Em uma B+, os nós internos contêm:
 - A) Dados completos
 - B) Referências para folhas
 - C) Chaves de indexação
 - D) Dados duplicados
4. Como as folhas se relacionam em uma B+?
 - A) Não se relacionam
 - B) Encadeamento duplo
 - C) Encadeamento simples
 - D) Ordenação aleatória
5. O que ocorre na remoção de uma chave em uma árvore B+?
 - A) Remoção direta
 - B) Reestruturação dos nós internos
 - C) Reorganização das folhas
 - D) Todas as anteriores

3. Árvore Rubro-Negra

Características:

- Árvore binária de busca balanceada.
- Cada nó tem uma cor (vermelho ou preto).
- Regras mantêm balanceamento sem exigir rebalanceamento completo.

Exemplo de Inserção:

Inserir 10 → raiz preta

Inserir 20 → vermelho

Inserir 30 → cria conflito → rotação à esquerda + recoloração

Remove como em BST → corrige cores e faz rotações para manter propriedades.

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

1. Uma árvore rubro-negra é uma:
 - A) Heap
 - B) Trie
 - C) Árvore balanceada

D) Árvore AVL

2. Quantas cores um nó pode ter?

- A) 1
- B) 2
- C) 3
- D) 4

3. A raiz de uma árvore rubro-negra sempre é:

- A) Vermelha
- B) Nula
- C) Preta
- D) Alternada

4. Quando duas inserções consecutivas criam dois nós vermelhos, o que é necessário?

- A) Nada
- B) Recoloração e rotação
- C) Apagar o nó
- D) Dividir a árvore

4. Trie (Árvore de Prefixos)

Características:

- Árvore para busca de strings por prefixo.
- Cada nível representa uma letra.
- Usada em dicionários, autocomplete e compressão.

Exemplo de Inserção:

Inserir "GATO", "GALO", "GATA":

- $G \rightarrow A \rightarrow T \rightarrow O$
 $\rightarrow A$
 $\rightarrow L \rightarrow O$

<https://www.cs.usfca.edu/~galles/visualization/Trie.html>

Buscar "GATO" → percorre nós $G \rightarrow A \rightarrow T \rightarrow O \rightarrow$ verifica se é palavra completa.

Remover "GALO" → marca como não-palavra ou remove ramos desnecessários.

1. Qual o uso típico de uma Trie?
 - A) Ordenação de inteiros
 - B) Autocomplete
 - C) Árvores de decisão
 - D) Heap sort

2. Cada nível da Trie representa:
 - A) Um caractere
 - B) Uma palavra
 - C) Um nó pai
 - D) Um índice

3. Tries são úteis quando:
 - A) Não há repetição
 - B) Há muitos prefixos em comum
 - C) Há muitos números inteiros
 - D) Árvores forem balanceadas

4. O que acontece ao remover uma palavra de uma Trie?
 - A) Toda a árvore é removida
 - B) Somente a última letra
 - C) Marca o fim como falso e/ou remove ramos desnecessários
 - D) Nada

5. Heap (Min-Heap e Max-Heap)

Características:

- Árvore binária completa.
- Mantém propriedade de heap:
 - Min-Heap: $\text{pai} \leq \text{filhos}$
 - Max-Heap: $\text{pai} \geq \text{filhos}$
- Usada em filas de prioridade, heapsort.

Exemplo de Inserção:

<https://www.cs.usfca.edu/~galles/visualization/Heap.html>

Min-Heap: Inserir 3, 5, 7, 1 → reordena para manter o menor no topo.

Busca pelo menor (min-heap) ou maior (max-heap) sempre na raiz.

Remove a raiz → substitui pelo último → faz heapify.

1. A raiz de um Min-Heap contém:
 - A) O maior elemento
 - B) O menor elemento
 - C) Um valor aleatório
 - D) Nenhum elemento
2. Heap é uma:
 - A) Árvore AVL
 - B) Árvore rubro-negra
 - C) Árvore binária completa
 - D) Trie
3. O heap é usado em:
 - A) Busca binária
 - B) Autocomplete
 - C) Ordenação (heapsort)
 - D) Árvores B
4. Heapify é:
 - A) Processo de ordenação por seleção
 - B) Rebalanceamento do heap
 - C) Conversão de array em trie
 - D) Remoção de prefixos

6. Árvore Splay

Características:

- Árvore binária de busca autobalanceada.
- Nó acessado é movido para a raiz via rotações (splay).
- Otimiza acessos frequentes.

Exemplo de Inserção:

<https://www.cs.usfca.edu/~galles/visualization/SplayTree.html>

Inserir 10, 20, 30 → após inserir 30, ele é rotacionado até virar a raiz.

Buscar 10 → faz splay de 10 até a raiz.

Remove normalmente → o nó anterior é rotacionado para a raiz.

1. O que ocorre após acessar um nó em uma Splay Tree?
 - A) Ele é deletado
 - B) Vai para a folha
 - C) Vai para a raiz
 - D) Nada
2. Vantagem da Splay Tree:
 - A) Não precisa de balanceamento
 - B) Favorece acessos frequentes
 - C) Armazena strings
 - D) Permite ordenação
3. O que é operação de splay?
 - A) Ordenação
 - B) Rotação até raiz
 - C) Divisão de árvore
 - D) Heapify
4. Inserção em Splay Tree pode causar:
 - A) Rebalanceamento
 - B) Splay automático
 - C) Remoção da raiz
 - D) Nada

Treinando para prova:

1. Árvore B (ordem 3)

Dada uma árvore B de ordem 3 (cada nó pode conter no máximo 2 chaves e 3 filhos), realize as seguintes operações na ordem em que são apresentadas:

1. **Inserir** os valores: 10, 20, 5, 6, 12, 30, 7, 17
2. Mostrar a árvore após todas as inserções.

3. **Remover** os valores: 6, 17, 10
4. Mostrar a árvore após cada remoção, destacando qualquer fusão ou redistribuição de chaves.

2. Árvore B+ (ordem 3)

Considere uma árvore B+ de ordem 3. Insira os seguintes elementos e represente a estrutura da árvore após cada inserção:

1. **Inserir:** 15, 25, 5, 10, 20, 30, 35
2. Mostrar o encadeamento das folhas.
3. **Remover** os valores: 10, 25
4. Atualize a estrutura da árvore após cada remoção, mantendo a propriedade de encadeamento entre folhas.

3. Árvore Rubro-Negra

Construa uma árvore rubro-negra inserindo os seguintes valores na ordem dada:

1. **Inserir:** 30, 15, 60, 7, 22, 45, 75, 17
2. Mostre as cores e a estrutura da árvore após cada inserção.
3. **Remover:** 22, 15
4. Após cada remoção, indique as rotações e recolorações necessárias.

4. Trie (com strings)

Construa uma Trie com as seguintes palavras:

1. **Inserir:** "bola", "boca", "bobo", "barco", "bala", "banda"
2. Mostre a estrutura da árvore após todas as inserções (nó por letra).
3. **Remover:** "bobo", "barco"
4. Apresente a estrutura da Trie após cada remoção, explicando se algum nó intermediário foi removido.

5. Heap (Min-Heap)

Utilize um Min-Heap (heap com o menor valor na raiz). Execute:

1. **Inserir** os valores: 9, 5, 12, 3, 8, 1, 6
2. Mostre o array correspondente ao heap após cada inserção.
3. **Remover**: três vezes o menor valor (isto é, executar `removeMin()` três vezes).
4. Atualize a estrutura do heap (array) após cada remoção.

6. Árvore Splay

Construa uma árvore Splay inserindo os seguintes elementos:

1. **Inserir**: 40, 20, 60, 10, 30, 50, 70
2. Após cada inserção, realize a operação de splay (colocar o novo elemento na raiz).
3. **Buscar**: 30 → aplique splay para movê-lo à raiz.
4. **Remover**: 60
5. Mostre a estrutura da árvore após a remoção, lembrando que o último acesso também provoca splay.